Show that $\ln(1 - x) \geq -x - x^2$ when $0 < x \leq 1/2$. (*Hint:* Start with equation (33.13), group all the terms after the first three, and use equation (A.7) on page 1142.)

33.2-3
Consider a randomized variant of the algorithm given in the proof of Lemma 33.3, in which some expert never makes a mistake. At each step, choose an expert E_i uniformly at random from the set S and then make the same predication as E_i. Show that the expected number of mistakes made by this algorithm is $\lceil \lg n \rceil$.

33.2-4
Consider a randomized version of WEIGHTED-MAJORITY. The algorithm is the same, except for the prediction step, which interprets the weights as a probability distribution over the experts and chooses an expert E_i according to that distribution. It then chooses its prediction to be the same as the prediction made by expert E_i. Show that, for any $0 < \epsilon < 1/2$, the expected number of mistakes made by this algorithm is at most $(1 + \epsilon)m^* + (\ln n)/\epsilon$.

33.3 Gradient descent

Suppose that you have a set $\{p_1, p_2, \ldots, p_n\}$ of points and you want to find the line that best fits these points. For any line ℓ, there is a distance d_i between each point p_i and the line. You want to find the line that minimizes some function $f(d_1, \ldots, d_n)$. There are many possible choices for the definition of distance and for the function f. For example, the distance can be the projection distance to the line and the function can be the sum of the squares of the distances. This type of problem is common in data science and machine learning—the line is the hypothesis that best describes the data—where the particular definition of best is determined by the definition of distance and the objective f. If the definition of distance and the function f are linear, then we have a linear-programming problem, as discussed in Chapter 29. Although the

linear-programming framework captures several important problems, many other problems, including various machine-learning problems, have objectives and constraints that are not necessarily linear. We need frameworks and algorithms to solve such problems.

In this section, we consider the problem of optimizing a continuous function and discuss one of the most popular methods to do so: gradient descent. Gradient descent is a general method for finding a local minimum of a function $f : \mathbb{R}^n \to \mathbb{R}$, where informally, a local minimum of a function f is a point $\mathbf{x}$ for which $f(\mathbf{x}) \le f(\mathbf{x}')$ for all $\mathbf{x}'$ that are "near" $\mathbf{x}$. When the function is convex, it can find a point near the ***global minimizer*** of f: an n-vector argument $\mathbf{x} = (x_1, x_2, \ldots, x_n)$ such that $f(\mathbf{x})$ is minimum. For the intuitive idea behind gradient descent, imagine being in a landscape of hills and valleys, and wanting to get to a low point as quickly as possible. You survey the terrain and choose to move in the direction that takes you downhill the fastest from your current position. You move in that direction, but only for a short while, because as you proceed, the terrain changes and you might need to choose a different direction. So you stop, reevaluate the possible directions and move another short distance in the steepest downhill direction, which might differ from the direction of your previous movement. You continue this process until you reach a point from which all directions lead up. Such a point is a local minimum.

In order to make this informal procedure more formal, we need to define the gradient of a function, which in the analogy above is a measure of the steepness of the various directions. Given a function $f : \mathbb{R}^n \to \mathbb{R}$, its ***gradient*** ∇f is a function $\nabla f : \mathbb{R}^n \to \mathbb{R}^n$ comprising n partial derivatives: $(\nabla f)(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots, \frac{\partial f}{\partial x_n}\right)$. Analogous to the derivative of a function of a single variable, the gradient can be viewed as a direction in which the function value locally increases the fastest, and the rate of that increase. This view is informal; in order to make it formal we would have to define what local means and place certain conditions, such as continuity or existence of derivatives, on the function. Nevertheless, this view motivates the key step of gradient descent—move in the direction

opposite to the gradient, by a distance influenced by the magnitude of the gradient.

The general procedure of gradient descent proceeds in steps. You start at some initial point $\mathbf{x^{(0)}}$, which is an n-vector. At each step t, you compute the value of the gradient of f at point $\mathbf{x^{(t)}}$, that is, $(\nabla f)(\mathbf{x^{(t)}})$, which is also an n-vector. You then move in the direction opposite to the gradient in each dimension at $\mathbf{x^{(t)}}$ to arrive at the next point $\mathbf{x^{(t+1)}}$, which again is an n-vector. Because you moved in a monotonically decreasing direction in each dimension, you should have that $f(\mathbf{x^{(t+1)}}) \leq f(\mathbf{x^{(t)}})$. Several details are needed to turn this idea into an actual algorithm. The two main details are that you need an initial point and that you need to decide how far to move in the direction of the negative gradient. You also need to understand when to stop and what you can conclude about the quality of the solution found. We will explore these issues further in this section, for both constrained minimization, where there are additional constraints on the points, and unconstrained minimization, where there are none.

Unconstrained gradient descent

In order to gain intuition, let's consider unconstrained gradient descent in just one dimension, that is, when f is a function of a scalar x, so that $f : \mathbb{R} \rightarrow \mathbb{R}$. In this case, the gradient ∇f of f is just $f'(x)$, the derivative of f with respect to x. Consider the function f shown in blue in Figure 33.3, with minimizer x^* and starting point $x^{(0)}$. The gradient (derivative) $f'(x^{(0)})$, shown in orange, has a negative slope, so that a small step from $x^{(0)}$ in the direction of increasing x results in a point x' for which $f(x') < f(x^{(0)})$. Too large a step, however, results in a

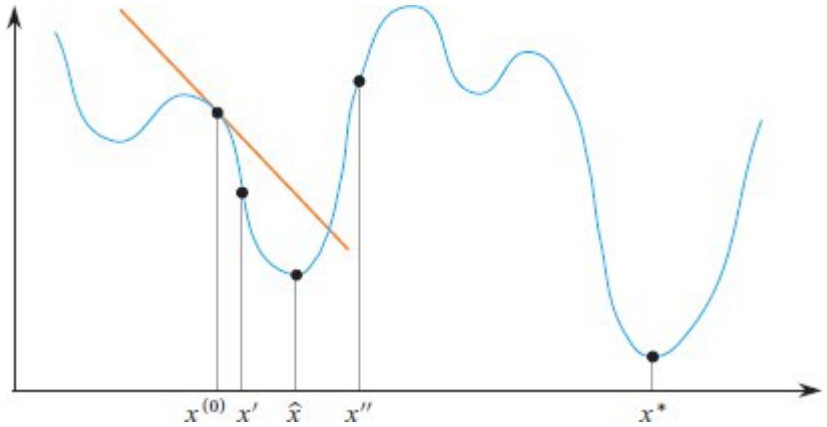

Figure 33.3 A function $f : \mathbb{R} \to \mathbb{R}$, shown in blue. Its gradient at point $x^{(0)}$, in orange, has a negative slope, and so a small increase in x from $x^{(0)}$ to x' results in $f(x') < f(x^{(0)})$. Small increases in x from $x^{(0)}$ head toward $\hat{x}$, which gives a local minimum. Too large an increase in x can end up at x'', where $f(x'') > f(x^{(0)})$. Small steps starting from $x^{(0)}$ and going only in the direction of decreasing values of f cannot end up at the global minimizer x^*.

point x'' for which $f(x'') > f(x^{(0)})$, so this is a bad idea. Restricting ourselves to small steps, where each one has $f(x') < f(x)$, eventually results in getting close to point $\hat{x}$, which gives a local minimum. By taking only small downhill steps, however, gradient descent has no chance to get to the global minimizer x^*, given the starting point $x^{(0)}$.

We draw two observations from this simple example. First, gradient descent converges toward a local minimum, and not necessarily a global minimum. Second, the speed at which it converges and how it behaves are related to properties of the function, to the initial point, and to the step size of the algorithm.

The procedure GRADIENT-DESCENT on the facing page takes as input a function f, an initial point $\mathbf{x^{(0)}} \in \mathbb{R}^n$, a fixed step-size multiplier $\gamma > 0$, and a number $T > 0$ of steps to take. Each iteration of the **for** loop of lines 2–4 performs a step by computing the n-dimensional gradient at point $\mathbf{x^{(t)}}$ and then moving distance γ in the opposite direction in the n-dimensional space. The complexity of computing the gradient depends on the function f and can sometimes be expensive. Line 3 sums the points visited. After the loop terminates, line 6 returns

x-avg, the average of all the points visited except for the last one, $\mathbf{x}^{(T)}$. It might seem more natural to return $\mathbf{x}^{(T)}$, and in fact, in many circumstances, you might prefer to have the function return $\mathbf{x}^{(T)}$. For the version we will analyze, however, we use **x-avg**.

```
GRADIENT-DESCENT(f, x^(0), γ, T)
1  sum = 0                                  // n-dimensional vector, initially all 0
2  for t = 0 to T – 1
3      sum = sum + x^(t)                    // add each of n dimensions into sum
4      x^(t+1) = x^(t) – γ · (∇f)(x^(t))    // (∇f)(x^(t)), x^(t+1) are n-dimensional
5  x-avg = sum/T                            // divide each of n dimensions by T
6  return x-avg
```

Figure 33.4 depicts how gradient descent ideally runs on a convex 1-dimensional function.[1] We'll define convexity more formally below, but the figure shows that each iteration moves in the direction opposite to the gradient, with the distance moved being proportional to the magnitude of the gradient. As the iterations proceed, the magnitude of the gradient decreases, and thus the distance moved along the horizontal axis decreases. After each iteration, the distance to the optimal point $\mathbf{x}^*$ decreases. This ideal behavior is not guaranteed to occur in general, but the analysis in the remainder of this section formalizes when this behavior occurs and quantifies the number of iterations needed. Gradient descent does not always work, however. We have already seen that if the function is not convex, gradient descent can converge to a local, rather than global, minimum. We have also seen that if the step size is too large, GRADIENT-DESCENT can overshoot the minimum and wind up farther away. (It is also possible to overshoot the minimum and wind up closer to the optimum.)

Analysis of unconstrained gradient descent for convex functions

Our analysis of gradient descent focuses on convex functions. Inequality (C.29) on page 1194 defines a convex function of one variable, as shown in Figure 33.5. We can extend that definition to a function $f : \mathbb{R}^n \to \mathbb{R}$ and say that f is ***convex*** if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and for all $0 \le \lambda \le 1$, we have

$$f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \le \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}) . \tag{33.18}$$

(Inequalities (33.18) and (C.29) are the same, except for the dimensions of $\mathbf{x}$ and $\mathbf{y}$.) We also assume that our convex functions are closed[2] and differentiable.

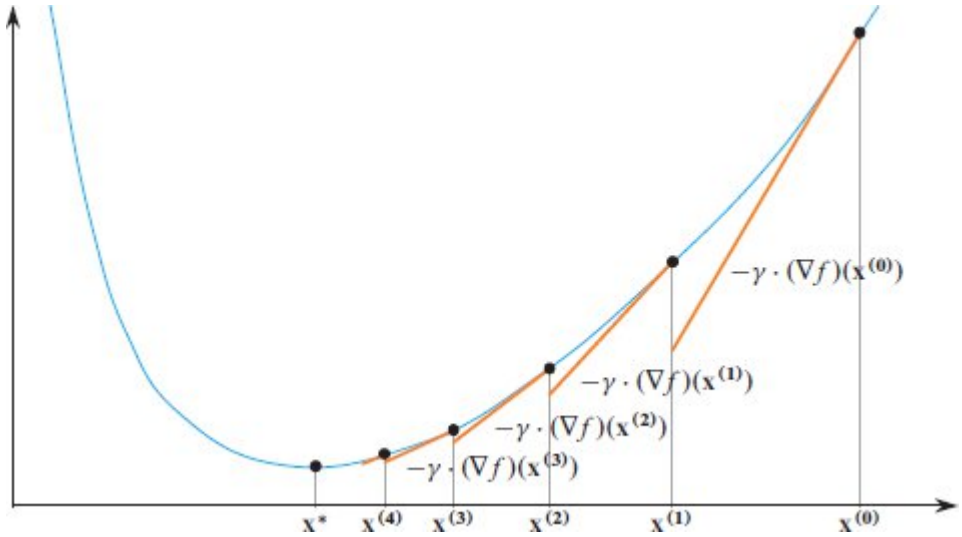

Figure 33.4 An example of running gradient descent on a convex function $f : \mathbb{R} \to \mathbb{R}$, shown in blue. Beginning at point $\mathbf{x}^{(0)}$, each iteration moves in the direction opposite to the gradient, and the distance moved is proportional to the magnitude of the gradient. Orange lines represent the negative of the gradient at each point, scaled by the step size γ. As the iterations proceed, the magnitude of the gradient decreases, and the distance moved decreases correspondingly. After each iteration, the distance to the optimal point $\mathbf{x}^*$ decreases.

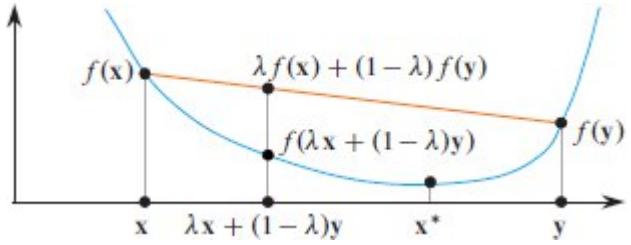

Figure 33.5 A convex function $f : \mathbb{R} \to \mathbb{R}$, shown in blue, with local and global minimizer $\mathbf{x}^*$. Because f is convex, $f(\lambda\mathbf{x} + (1 - \lambda)\mathbf{y}) \le \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ for any two values $\mathbf{x}$ and $\mathbf{y}$ and all $0 \le \lambda \le 1$, shown for a particular value of λ. Here, the orange line segment represents all values $\lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ for $0 \le \lambda \le 1$, and it is above the blue line.

A convex function has the property that any local minimum is also a global minimum. To verify this property, consider inequality (33.18), and suppose for the purpose of contradiction that $\mathbf{x}$ is a local minimum but not a global minimum and $\mathbf{y} \ne \mathbf{x}$ is a global minimum, so $f(\mathbf{y}) < f(\mathbf{x})$. Then we have

$$\begin{aligned} f(\lambda\mathbf{x} + (1 - \lambda)\mathbf{y}) &\le \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}) \quad \text{(by inequality (33.18))} \\ &< \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{x}) \\ &= f(\mathbf{x}). \end{aligned}$$

Thus, letting approach 1, we see that there is another point near $\mathbf{x}$, say $\mathbf{x}'$, such that $f(\mathbf{x}') < f(\mathbf{x})$, so $\mathbf{x}$ is not a local minimum.

Convex functions have several useful properties. The first property, whose proof we leave as Exercise 33.3-1, says that a convex function always lies above its tangent hyperplane. In the context of gradient descent, angle brackets denote the notation for inner product defined on page 1219 rather than denoting a sequence.

Lemma 33.6

For any convex differentiable function $f : \mathbb{R}^n \to \mathbb{R}$ and for all $x, y \in \mathbb{R}^n$, we have $\le f(\mathbf{x}) \le f(\mathbf{y}) + \langle(\nabla f)(\mathbf{x}), \mathbf{x} - \mathbf{y}\rangle$.

■

The second property, which Exercise 33.3-2 asks you to prove, is a repeated application of the definition of convexity in inequality (33.18).

Lemma 33.7

For any convex function $f : \mathbb{R}^n \to \mathbb{R}$, for any integer $T \geq 1$, and for all $\mathbf{x}^{(0)}, \ldots, \mathbf{x}^{(T-1)} \in \mathbb{R}^n$, we have

$$f\left(\frac{\mathbf{x}^{(0)} + \cdots + \mathbf{x}^{(T-1)}}{T}\right) \leq \frac{f(\mathbf{x}^{(0)}) + \cdots + f(\mathbf{x}^{(T-1)})}{T} . \tag{33.19}$$

■

The left-hand side of inequality (33.19) is the value of f at the vector **x-avg** that GRADIENT-DESCENT returns.

We now proceed to analyze GRADIENT-DESCENT. It might not return the exact global minimizer $\mathbf{x}^*$. We use an error bound ϵ, and we want to choose T so that $f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) \leq \epsilon$ at termination. The value of ϵ depends on the number T of iterations and two additional values. First, since you expect it to be better to start close to the global minimizer, ϵ is a function of

$$R = \|\mathbf{x}^{(0)} - \mathbf{x}^*\| , \tag{33.20}$$

the euclidean norm (or distance, defined on page 1219) of the difference between $\mathbf{x}^{(0)}$ and $\mathbf{x}^*$. The error bound ϵ is also a function of a quantity we call L, which is an upper bound on the magnitude $\|(\nabla f)(\mathbf{x})\|$ of the gradient, so that

$$\|(\nabla f)(\mathbf{x})\| \leq L , \tag{33.21}$$

where $\mathbf{x}$ ranges over all the points $\mathbf{x}^{(0)}, \ldots, \mathbf{x}^{(T-1)}$ whose gradients are computed by GRADIENT-DESCENT. Of course, we don't know the values of L and R, but for now let's assume that we do. We'll discuss later how to remove these assumptions. The analysis of GRADIENT-DESCENT is summarized in the following theorem.

Theorem 33.8

Let $\mathbf{x}^* \in \mathbb{R}^n$ be the minimizer of a convex function f, and suppose that an execution of GRADIENT-DESCENT(f, $\mathbf{x}^{(0)}$, γ, T) returns **x-avg**,

where $\gamma = R/(L\sqrt{T})$ and R and L are defined in equations (33.20) and (33.21). Let $\epsilon = RL/\sqrt{T}$. Then we have $f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) \le \epsilon$.

■

We now prove this theorem. We do not give an absolute bound on how much progress each iteration makes. Instead, we use a potential function, as in Section 16.3. Here, we define a potential $\Phi(t)$ after computing $\mathbf{x}^{(t)}$, such that $\Phi(t) \ge 0$ for $t = 0, \ldots, T$. We define the ***amortized progress*** in the iteration that computes $\mathbf{x}^{(t)}$ as

$$p(t) = f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) + \Phi(t+1) - \Phi(t) . \tag{33.22}$$

Along with including the change in potential ($\Phi(t+1) - \Phi(t)$), equation (33.22) also subtracts the minimum value $f(\mathbf{x}^*)$ because ultimately, you care not about the values $f(\mathbf{x}^{(t)})$ but about how close they are to $f(\mathbf{x}^*)$. Suppose that we can show that $p(t) \le B$ for some value B and $t = 0, \ldots, T-1$. Then we can substitute for $p(t)$ using equation (33.22), giving

$$f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) \le B - \Phi(t+1) + \Phi(t) . \tag{33.23}$$

Summing inequality (33.23) over $t = 0, \ldots, T-1$ yields

$$\sum_{t=0}^{T-1} (f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*)) \le \sum_{t=0}^{T-1} (B - \Phi(t+1) + \Phi(t)) .$$

Observing that we have a telescoping series on the right and regrouping terms, we have that

$$\left(\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)}) \right) - T \cdot f(\mathbf{x}^*) \le TB - \Phi(T) + \Phi(0) .$$

Dividing by T and dropping the positive term $\Phi(T)$ gives

$$\frac{\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)})}{T} - f(\mathbf{x}^*) \le B + \frac{\Phi(0)}{T} , \tag{33.24}$$

and thus we have

$$
\begin{aligned}
f(\mathbf{x}\text{-avg}) - f(\mathbf{x}^*) &= f\left(\frac{\sum_{t=0}^{T-1} \mathbf{x}^{(t)}}{T}\right) - f(\mathbf{x}^*) && \text{(by the definition of } \mathbf{x}\text{-avg)} \\
&\le \frac{\sum_{t=0}^{T-1} f(\mathbf{x}^{(t)})}{T} - f(\mathbf{x}^*) && \text{(by Lemma 33.7)} \\
&\le B + \frac{\Phi(0)}{T} && \text{(by inequality (33.24))} . \quad (33.25)
\end{aligned}
$$

In other words, if we can show that $p(t) \le B$ for some value B and choose a potential function where $\Phi(0)$ is not too large, then inequality (33.25) tells us how close the function value $f(\mathbf{x}\text{-avg})$ is to the function value $f(\mathbf{x}^*)$ after T iterations. That is, we can set the error bound ϵ to $B + \Phi(0)/T$.

In order to bound the amortized progress, we need to come up with a concrete potential function. Define the potential function $\Phi(t)$ by

$$
\Phi(t) = \frac{\left\|\mathbf{x}^{(t)} - \mathbf{x}^*\right\|^2}{2\gamma} , \tag{33.26}
$$

that is, the potential function is proportional to the square of the distance between the current point and the minimizer $\mathbf{x}^*$. With this potential function in hand, the next lemma provides a bound on the amortized progress made in any iteration of GRADIENT-DESCENT.

Lemma 33.9

Let $\mathbf{x}^* \in \mathbb{R}^n$ be the minimizer of a convex function f, and consider an execution of GRADIENT-DESCENT(f, $\mathbf{x}^{(0)}$, γ, T). Then for each point $\mathbf{x}^{(t)}$ computed by the procedure, we have that

$$
p(t) = f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) + \Phi(t+1) - \Phi(t) \le \frac{\gamma L^2}{2} .
$$

Proof We first bound the potential change $\Phi(t + 1) - \Phi(t)$. Using the definition of $\Phi(t)$ from equation (33.26), we have

$$
\Phi(t+1) - \Phi(t) = \frac{1}{2\gamma}\left\|\mathbf{x}^{(t+1)} - \mathbf{x}^*\right\|^2 - \frac{1}{2\gamma}\left\|\mathbf{x}^{(t)} - \mathbf{x}^*\right\|^2 . \tag{33.27}
$$

From line 4 in GRADIENT-DESCENT, we know that

$$
\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)} = -\gamma \cdot (\nabla f)(\mathbf{x}^{(t)}) , \tag{33.28}
$$

and so we would like to rewrite equation (33.27) to have $\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}$ terms. As Exercise 33.3-3 asks you to prove, for any two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, we have

$$\|\mathbf{a} + \mathbf{b}\|^2 - \|\mathbf{a}\|^2 = 2\langle \mathbf{b}, \mathbf{a} \rangle + \|\mathbf{b}\|^2 \ . \tag{33.29}$$

Letting $\mathbf{a} = \mathbf{x}^{(t)} - \mathbf{x}^*$ and $\mathbf{b} = \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}$, we can write the right-hand side of equation (33.27) as $\frac{1}{2\gamma}\left(\|\mathbf{a} + \mathbf{b}\|^2 - \|\mathbf{a}\|^2\right)$. Then we can express the potential change as

$$\begin{aligned}
&\Phi(t+1) - \Phi(t) \\
&= \frac{1}{2\gamma}\left\|\mathbf{x}^{(t+1)} - \mathbf{x}^*\right\|^2 - \frac{1}{2\gamma}\left\|\mathbf{x}^{(t)} - \mathbf{x}^*\right\|^2 && \text{(by equation (33.27))} \\
&= \frac{1}{2\gamma}\left(2\langle \mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}, \mathbf{x}^{(t)} - \mathbf{x}^* \rangle + \left\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\right\|^2\right) && \text{(by equation (33.29))} \\
&= \frac{1}{2\gamma}\left(2\langle -\gamma \cdot (\nabla f)(\mathbf{x}^{(t)}), \mathbf{x}^{(t)} - \mathbf{x}^* \rangle + \left\|-\gamma \cdot (\nabla f)(\mathbf{x}^{(t)})\right\|^2\right) \\
& && \text{(by equation (33.28))} \\
&= -\langle (\nabla f)(\mathbf{x}^{(t)}), \mathbf{x}^{(t)} - \mathbf{x}^* \rangle + \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 && (33.30) \\
& && \text{(by equation (D.3) on page 1219)} \\
&\le -(f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*)) + \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 && \text{(by Lemma 33.6)} \ ,
\end{aligned}$$

and thus we have

$$\Phi(t+1) - \Phi(t) \le -(f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*)) + \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 \tag{33.31}$$

We can now proceed to bound $p(t)$. By the bound on the potential change from inequality (33.31), and using the definition of L (inequality (33.21)), we have

$$\begin{aligned}
p(t) &= f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) + \Phi(t+1) - \Phi(t) && \text{(by equation (33.22))} \\
&\le f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*) - (f(\mathbf{x}^{(t)}) - f(\mathbf{x}^*)) + \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 \\
&= \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 && \text{(by inequality (33.31))} \\
&\le \frac{\gamma L^2}{2} && \text{(by inequality (33.21))} \ .
\end{aligned}$$

■

sult in the following theorem

Having bounded the amortized progress in one step, we now analyze the entire GRADIENT-DESCENT procedure, completing the proof of Theorem 33.8.

Proof of Theorem 33.8 Inequality (33.25) tells us that if we have an upper bound of B for $p(t)$, then we also have the bound $f(\mathbf{x\text{-}avg}) - f(\mathbf{x}^*) \le B + \Phi(0)/T$. By equations (33.20) and (33.26), we have that $\Phi(0) = R^2/(2\gamma)$. Lemma 33.9 gives us the upper bound of $B = \gamma L^2/2$, and so we have

$$\begin{aligned} f(\mathbf{x\text{-}avg}) - f(\mathbf{x}^*) &\le B + \frac{\Phi(0)}{T} \qquad \text{(by inequality (33.25))} \\ &= \frac{\gamma L^2}{2} + \frac{R^2}{2\gamma T} . \end{aligned}$$

Our choice of $\gamma = R/(L\sqrt{T})$ in the statement of Theorem 33.8 balances the two terms, and we obtain

$$\begin{aligned} \frac{\gamma L^2}{2} + \frac{R^2}{2\gamma T} &= \frac{R}{L\sqrt{T}} \cdot \frac{L^2}{2} + \frac{R^2}{2T} \cdot \frac{L\sqrt{T}}{R} \\ &= \frac{RL}{2\sqrt{T}} + \frac{RL}{2\sqrt{T}} \\ &= \frac{RL}{\sqrt{T}} . \end{aligned}$$

Since we chose $\epsilon = RL/\sqrt{T}$ in the theorem statement, the proof is complete.

■

Continuing under the assumption that we know R (from equation (33.20)) and L (from inequality (33.21)), we can think of the analysis in a slightly different way. We can presume that we have a target accuracy ϵ and then compute the number of iterations needed. That is, we can solve $\epsilon = RL/\sqrt{T}$ for T, obtaining $T = R^2L^2/\epsilon^2$. The number of iterations thus depends on the square of R and L and, most importantly, on $1/\epsilon^2$. (The definition of L from inequality (33.21) depends on T, but we may know an upper bound on L that doesn't

depend on the particular value of T.) Thus, if you want to halve your error bound, you need to run four times as many iterations.

It is quite possible that we don't really know R and L, since you'd need to know $\mathbf{x}^*$ in order to know R (since $R = \|\mathbf{x}^{(0)} - \mathbf{x}^*\|$), and you might not have an explicit upper bound on the gradient, which would provide L. You can, however, interpret the analysis of gradient descent as a proof that there is some step size for which the procedure makes progress toward the minimum. You can then compute a step size for which $f(\mathbf{x}^{(t)}) - f(\mathbf{x}^{(t+1)})$ is large enough. In fact, not having a fixed step size multiplier can actually help in practice, as you are free to use any step size s that achieves sufficient decrease in the value of f. You can search for a step size that achieves a large decrease via a binary-search-like routine, which is often called ***line search***. For a given function f and step size s, define the function $g(\mathbf{x}^{(t)}, s) = f(\mathbf{x}^{(t)}) - s(\nabla f)(\mathbf{x}^{(t)})$. Start with a small step size s for which $g(\mathbf{x}^{(t)}, s) \le f(\mathbf{x}^{(t)})$. Then repeatedly double s until $g(\mathbf{x}^{(t)}, 2s) \ge g(\mathbf{x}^{(t)}, s)$, and then perform a binary search in the interval $[s, 2s]$. This procedure can produce a step size that achieves a significant decrease in the objective function. In other circumstances, however, you may know good upper bounds on R and L, typically from problem-specific information, which can suffice.

The dominant computational step in each iteration of the **for** loop of lines 2–4 is computing the gradient. The complexity of computing and evaluating a gradient varies widely, depending on the application at hand. We'll discuss several applications later.

Constrained gradient descent

We can adapt gradient descent for constrained minimization to minimize a closed convex function $f(\mathbf{x})$, subject to the additional requirement that $\mathbf{x} \in K$, where K is a closed convex body. A ***body*** $K \subseteq \mathbb{R}^n$ is ***convex*** if for all $\mathbf{x}, \mathbf{y} \in K$, the convex combination $\lambda\mathbf{x}+(1-\lambda)\mathbf{y} \in K$ for all $0 \le \lambda \le 1$. A ***closed*** convex body contains its limit points. Somewhat surprisingly, restricting to the constrained problem does not significantly increase the number of iterations of gradient descent. The

idea is that you run the same algorithm, but in each iteration, check whether the current point $\mathbf{x}^{(t)}$ is still within the convex body K. If it is not, just move to the closest point in K. Moving to the closest point is known as ***projection***. We formally define the projection $\Pi_K(\mathbf{x})$ of a point $\mathbf{x}$ in n dimensions onto a convex body K as the point $\mathbf{y} \in K$ such that $\|\mathbf{x} - \mathbf{y}\| = \min\{\|\mathbf{x} - \mathbf{z}\| : z \in K\}$. If we have $\mathbf{x} \in K$, then $\Pi_K(\mathbf{x}) = \mathbf{x}$.

This one change yields the procedure GRADIENT-DESCENT-CONSTRAINED, in which line 4 of GRADIENT-DESCENT is replaced by two lines. It assumes that $\mathbf{x}^{(0)} \in K$. Line 4 of GRADIENT-DESCENT-CONSTRAINED moves in the direction of the negative gradient, and line 5 projects back onto K. The lemma that follows helps to show that when $\mathbf{x}^* \in K$, if the projection step in line 5 moves from a point outside of K to a point in K, it cannot be moving away from $\mathbf{x}^*$.

```
GRADIENT-DESCENT-CONSTRAINED(f, x^(0), γ, T, K)
1  sum = 0                                  // n-dimensional vector, initially all 0
2  for t = 0 to T – 1
3      sum = sum + x^(t)                    // add each of n dimensions into sum
4      x'^(t+1) = x^(t) – γ · (∇f)(x^(t))   // (∇f)(x^(t)), x'^(t+1) are n-dimensional
5      x^(t+1) = Π_K(x^(t+1))               // project onto K
6  x-avg = sum/T                            // divide each of n dimensions by T
7  return x-avg
```

Lemma 33.10

Consider a convex body $K \subseteq \mathbb{R}^n$ and points $\mathbf{a} \in K$ and $\mathbf{b}' \in \mathbb{R}^n$. Let $\mathbf{b} = \Pi_K(\mathbf{b}')$. Then $\|\mathbf{b} - \mathbf{a}\|^2 \le \|\mathbf{b}' - \mathbf{a}\|^2$.

Proof If $\mathbf{b}' \in K$, then $\mathbf{b} = \mathbf{b}'$ and the claim is true. Otherwise, $\mathbf{b}' \ne \mathbf{b}$, and as Figure 33.6 shows, we can extend the line segment between $\mathbf{b}$ and $\mathbf{b}'$ to a line ℓ. Let $\mathbf{c}$ be the projection of $\mathbf{a}$ onto ℓ. Point $\mathbf{c}$ may or may not be

in K, and if $\mathbf{a}$ is on the boundary of K, then $\mathbf{c}$ could coincide with $\mathbf{b}$. If $\mathbf{c}$ coincides with $\mathbf{b}$ (part (c) of the figure), then $\mathbf{abb}'$ is a right triangle, and so $\|\mathbf{b} - \mathbf{a}\|^2 \le \|\mathbf{b}' - \mathbf{a}\|^2$. If $\mathbf{c}$ does not coincide with $\mathbf{b}$ (parts (a) and (b) of the figure), then because of convexity, the angle $\angle\mathbf{abb}'$ must be obtuse. Because angle $\angle\mathbf{abb}'$ is obtuse, $\mathbf{b}$ lies between $\mathbf{c}$ and $\mathbf{b}'$ on ℓ. Furthermore, because $\mathbf{c}$ is the projection of $\mathbf{a}$ onto line ℓ, $\mathbf{acb}$ and $\mathbf{acb}'$ must be right triangles. By the Pythagorean theorem, we have that $\|\mathbf{b}' - \mathbf{a}\|^2 = \|\mathbf{a} - \mathbf{c}\|^2 + \|\mathbf{c} - \mathbf{b}'\|^2$ and $\|\mathbf{b} - \mathbf{a}\|^2 = \|\mathbf{a} - \mathbf{c}\|^2 + \|\mathbf{c} - \mathbf{b}\|^2$. Subtracting these two equations gives $\|\mathbf{b}' - \mathbf{a}\|^2 - \|\mathbf{b} - \mathbf{a}\|^2 = \|\mathbf{c} - \mathbf{b}'\|^2 - \|\mathbf{c} - \mathbf{b}\|^2$. Because $\mathbf{b}$ is between $\mathbf{c}$ and $\mathbf{b}'$, we must have $\|\mathbf{c} - \mathbf{b}'\|^2 \ge \|\mathbf{c} - \mathbf{b}\|^2$, and thus $\|\mathbf{b}' - \mathbf{a}\|^2 - \|\mathbf{b} - \mathbf{a}\|^2 \ge 0$. The lemma follows.

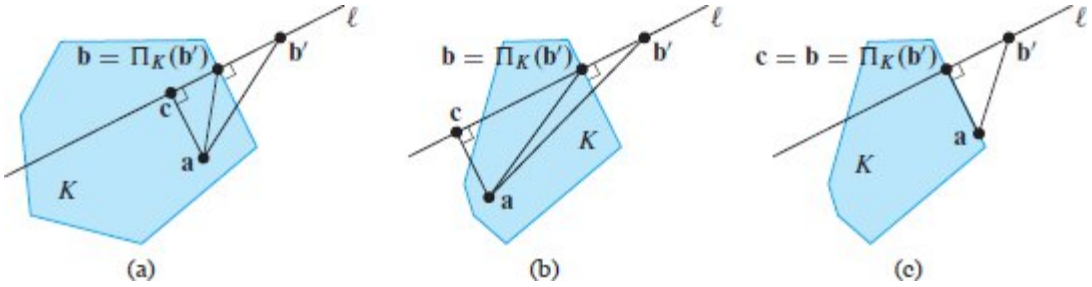

Figure 33.6 Projecting a point $\mathbf{b}'$ outside the convex body K to the closest point $\mathbf{b} = \Pi_K(\mathbf{b}')$ in K. Line ℓ is the line containing $\mathbf{b}$ and $\mathbf{b}'$, and point $\mathbf{c}$ is the projection of $\mathbf{a}$ onto ℓ. **(a)** When $\mathbf{c}$ is in K. **(b)** When $\mathbf{c}$ is not in K. **(c)** When $\mathbf{a}$ is on the boundary of K and $\mathbf{c}$ coincides with $\mathbf{b}$.

■

We can now repeat the entire proof for the unconstrained case and obtain the same bounds. Lemma 33.10 with $\mathbf{a} = \mathbf{x}^*$, $\mathbf{b} = \mathbf{x}^{(t+1)}$, and $\mathbf{b}' = \mathbf{x}'^{(t+1)}$ tells us that $\|\mathbf{x}^{(t+1)} - \mathbf{x}^*\|^2 \le \|\mathbf{x}'^{(t+1)} - \mathbf{x}^*\|^2$. We can therefore derive an upper bound that matches inequality (33.31). We continue to define $\Phi(t)$ as in equation (33.26), but noting that $\mathbf{x}^{(t+1)}$, computed in line 5 of GRADIENT-DESCENT-CONSTRAINED, has a different meaning here from in inequality (33.31):

$$
\begin{aligned}
&\Phi(t+1) - \Phi(t) \\
&\quad = \frac{1}{2\gamma}\left\|\mathbf{x}^{(t+1)} - \mathbf{x}^*\right\|^2 - \frac{1}{2\gamma}\left\|\mathbf{x}^{(t)} - \mathbf{x}^*\right\|^2 && \text{(by equation (33.27))} \\
&\quad \le \frac{1}{2\gamma}\left\|\mathbf{x}'^{(t+1)} - \mathbf{x}^*\right\|^2 - \frac{1}{2\gamma}\left\|\mathbf{x}^{(t)} - \mathbf{x}^*\right\|^2 && \text{(by Lemma 33.10)} \\
&\quad = \frac{1}{2\gamma}\left(2\langle \mathbf{x}'^{(t+1)} - \mathbf{x}^{(t)}, \mathbf{x}^{(t)} - \mathbf{x}^*\rangle + \left\|\mathbf{x}'^{(t+1)} - \mathbf{x}^*\right\|^2\right) \\
& && \text{(by equation (33.29))} \\
&\quad = \frac{1}{2\gamma}\left(2\langle -\gamma \cdot (\nabla f)(\mathbf{x}^{(t)}), \mathbf{x}^{(t)} - \mathbf{x}^*\rangle + \left\|-\gamma \cdot (\nabla f)(\mathbf{x}^{(t)})\right\|^2\right) \\
& && \text{(by line 4 of GRADIENT-DESCENT-CONSTRAINED)} \\
&\quad = -\langle (\nabla f)(\mathbf{x}^{(t)}), \mathbf{x}^{(t)} - \mathbf{x}^*\rangle + \frac{\gamma}{2}\left\|(\nabla f)(\mathbf{x}^{(t)})\right\|^2 .
\end{aligned}
$$

With the same upper bound on the change in the potential function as in equation (33.30), the entire proof of Lemma 33.9 can proceed as before. We can therefore conclude that the procedure GRADIENT-DESCENT-CONSTRAINED has the same asymptotic complexity as GRADIENT-DESCENT. We summarize this result in the following theorem.

Theorem 33.11

Let $K \subseteq \mathbb{R}^n$ be a convex body, $\mathbf{x}^* \in \mathbb{R}^n$ be the minimizer of a convex function f over K, and $\gamma = R/(L\sqrt{T})$, where R and L are defined in equations (33.20) and (33.21). Suppose that the vector **x-avg** is returned by an execution of GRADIENT-DESCENT-CONSTRAINED(f, $\mathbf{x}^{(0)}$, γ, T, K). Let $\epsilon = RL/\sqrt{T}$. Then we have $f(\textbf{x-avg}) - f(\mathbf{x}^*) \le \epsilon$.

■

Applications of gradient descent

Gradient descent has many applications to minimizing functions and is widely used in optimization and machine learning. Here we sketch how it can be used to solve linear systems. Then we discuss an application to machine learning: prediction using linear regression.

In Chapter 28, we saw how to use Gaussian elimination to solve a system of linear equations $A\mathbf{x} = \mathbf{b}$, thereby computing $\mathbf{x} = A^{-1}\mathbf{b}$. If A is an $n \times n$ matrix and $\mathbf{b}$ is a length-n vector, then the running time of

Gaussian elimination is $\Theta(n^3)$, which for large matrices might be prohibitively expensive. If an approximate solution is acceptable, however, you can use gradient descent.

First, let's see how to use gradient descent as a roundabout—and admittedly inefficient—way to solve for x in the scalar equation $ax = b$, where $a, x, b \in \mathbb{R}$. This equation is equivalent to $ax - b = 0$. If $ax - b$ is the derivative of a convex function $f(x)$, then $ax - b = 0$ for the value of x that minimizes $f(x)$. Given $f(x)$, gradient descent can then determine this minimizer. Of course, $f(x)$ is just the integral of $ax - b$, that is, $f(x) = \frac{1}{2}ax^2 - bx$, which is convex if $a \geq 0$. Therefore, one way to solve $ax = b$ for $a \geq 0$ is to find the minimizer for $\frac{1}{2}ax^2 - bx$ via gradient descent.

We now generalize this idea to higher dimensions, where using gradient descent may actually lead to a faster algorithm. One n-dimensional analog is the function $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T\mathbf{x}$, where A is an $n \times n$ matrix. The gradient of f with respect to $\mathbf{x}$ is the function $A\mathbf{x} - \mathbf{b}$. To find the value of $\mathbf{x}$ that minimizes f, we set the gradient of f to 0 and solve for $\mathbf{x}$. Solving $A\mathbf{x}–\mathbf{b} = 0$ for $\mathbf{x}$, we obtain $\mathbf{x} = A^{-1}\mathbf{b}$, Thus, minimizing $f(\mathbf{x})$ is equivalent to solving $A\mathbf{x} = \mathbf{b}$. If $f(\mathbf{x})$ is convex, then gradient descent can approximately compute this minimum.

A 1-dimensional function is convex when its second derivative is positive. The equivalent definition for a multidimensional function is that it is convex when its Hessian matrix is positive-semidefinite (see page 1222 for a definition), where the ***Hessian matrix*** $(\nabla^2 f)(\mathbf{x})$ of a function $f(\mathbf{x})$ is the matrix in which entry (i, j) is the partial derivative of f with respect to i and j:

$$(\nabla^2 f)(\mathbf{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1 \partial x_1} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n \partial x_n} \end{pmatrix}.$$

Analogous to the 1-dimensional case, the Hessian of f is just A, and so if A is a positive-semidefinite matrix, then we can use gradient descent

to find a point $\mathbf{x}$ where $A\mathbf{x} \approx \mathbf{b}$. If R and L are not too large, then this method is faster than using Gaussian elimination.

Gradient descent in machine learning

As a concrete example of supervised learning for prediction, suppose that you want to predict whether a patient will develop heart disease. For each of m patients, you have n different attributes. For example, you might have $n = 4$ and the four pieces of data are age, height, blood pressure, and number of close family members with heart disease. Denote the data for patient i as a vector $\mathbf{x}^{(i)} \in \mathbb{R}^n$, with $x_j^{(i)}$ giving the jth entry in vector $\mathbf{x}^{(i)}$. The ***label*** of patient i is denoted by a scalar $y^{(i)} \in \mathbb{R}$, signifying the severity of the patient's heart disease. The hypothesis should capture a relationship between the $\mathbf{x}^{(i)}$ values and $y^{(i)}$. For this example, we make the modeling assumption that the relationship is linear, and therefore the goal is to compute the "best" linear relationship between the $\mathbf{x}^{(i)}$ values and $y^{(i)}$: a linear function $f : \mathbb{R}^n \to \mathbb{R}$ such that $f(\mathbf{x}^{(i)}) \approx y^{(i)}$ for each patient i. Of course, no such function may exist, but you would like one that comes as close as possible. A linear function f can be defined by a vector of weights $\mathbf{w} = (w_0, w_1, \ldots, w_n)$, with

$$f(\mathbf{x}) = w_0 + \sum_{j=1}^{n} w_j x_j \,. \tag{33.32}$$

When evaluating a machine-learning model, you need to measure how close each value $f(\mathbf{x}^{(i)})$ is to its corresponding label $y^{(i)}$. In this example, we define the error $e^{(i)} \in \mathbb{R}$ associated with patient i as $e^{(i)} = f(\mathbf{x}^{(i)}) - y^{(i)}$. The objective function we choose is to minimize the sum of squares of the errors, which is

$$\sum_{i=1}^{m} \left(e^{(i)}\right)^2 = \sum_{i=1}^{m} \left(f(\mathbf{x}^{(i)}) - y^{(i)}\right)^2$$
$$= \sum_{i=1}^{m} \left(w_0 + \sum_{j=1}^{n} w_j x_j^{(i)} - y^{(i)}\right)^2 . \tag{33.33}$$

The objective function is typically called the ***loss function***, and the ***least-squares error*** given by equation (33.33) is just one example of many possible loss functions. The goal is then, given the $\mathbf{x}^{(i)}$ and $y^{(i)}$ values, to compute the weights $w_0, w_1, \ldots, w_n$ so as to minimize the loss function in equation (33.33). The variables here are the weights $w_0, w_1, \ldots, w_n$ and not the $\mathbf{x}^{(i)}$ or $y^{(i)}$ values.

This particular objective is sometimes known as a ***least-squares fit***, and the problem of finding a linear function to fit data and minimize the least-squares error is called ***linear regression***. Finding a least-squares fit is also addressed in Section 28.3.

When the function f is linear, the loss function defined in equation (33.33) is convex, because it is the sum of squares of linear functions, which are themselves convex. Therefore, we can apply gradient descent to compute a set of weights to approximately minimize the least-squares error. The concrete goal of learning is to be able to make predictions on new data. Informally, if the features are all reported in the same units and are from the same range (perhaps from being normalized), then the weights tend to have a natural interpretation because the features of the data that are better predictors of the label have a larger associated weight. For example, you would expect that, after normalization, the weight associated with the number of family members with heart disease would be larger than the weight associated with height.

The computed weights form a model of the data. Once you have a model, you can make predictions, so that given new data, you can predict its label. In our example, given a new patient $\mathbf{x}'$ who is not part of the original training data set, you would still hope to predict the chance that the new patient develops heart disease. You can do so by computing the label $f(\mathbf{x}')$, incorporating the weights computed by gradient descent.

For this linear-regression problem, the objective is to minimize the expression in equation (33.33), which is a quadratic in each of the $n+1$ weights w_j. Thus, entry j in the gradient is linear in w_j. Exercise 33.3-5 asks you to explicitly compute the gradient and see that it can be computed in $O(nm)$ time, which is linear in the input size. Compared with the exact method of solving equation (33.33) in Chapter 28, which needs to invert a matrix, gradient descent is typically much faster.

Section 33.1 briefly discussed regularization—the idea that a complicated hypothesis should be penalized in order to avoid overfitting the training data. Regularization often involves adding a term to the objective function, but it can also be achieved by adding a constraint. One way to regularize this example would be to explicitly limit the norm of the weights, adding a constraint that $\|\mathbf{w}\| \le B$ for some bound $B > 0$. (Recall again that the components of the vector $\mathbf{w}$ are the variables in the present application.) Adding this constraint controls the complexity of the model, as the number of values w_j that can have large absolute value is now limited.

In order to run GRADIENT-DESCENT-CONSTRAINED for any problem, you need to implement the projection step, as well as to compute bounds on R and L. We conclude this section by describing these calculations for gradient descent with the constraint $\|\mathbf{w}\| \le B$. First, consider the projection step in line 5. Suppose that the update in line 4 results in a vector $\mathbf{w}'$. The projection is implemented by computing $\Pi_K(\mathbf{w}')$ where K is defined by $\|\mathbf{w}\| \le B$. This particular projection can be accomplished by simply scaling $\mathbf{w}'$, since we know that closest point in K to $\mathbf{w}'$ must be the point along the vector whose norm is exactly B. The amount z by which we need to scale $\mathbf{w}'$ to hit the boundary of K is the solution to the equation $z\,\|\mathbf{w}'\| = B$, which is solved by $z = B/\|\mathbf{w}'\|$. Hence line 5 is implemented by computing $\mathbf{w} = \mathbf{w}'B/\|\mathbf{w}'\|$. Because we always have $\|\mathbf{w}\| \le B$, Exercise 33.3-6 asks you to show that the upper bound on the magnitude L of the gradient is $O(B)$. We also get a bound on R, as follows. By the constraint $\|\mathbf{w}\| \le B$, we know that both $\|\mathbf{w}^{(0)}\| \le B$ and $\|\mathbf{w}^*\| \le B$, and thus $\|\mathbf{w}^{(0)} - \mathbf{w}^*\| \le 2B$. Using the definition of R in equation (33.20), we have $R = O(B)$. The

bound $RL/\sqrt{T}$ on the accuracy of the solution after T iterations in Theorem 33.11 becomes $O(B)L/\sqrt{T} = O(B^2/\sqrt{T})$.

Exercises

33.3-1
Prove Lemma 33.6. Start from the definition of a convex function given in equation (33.18). (*Hint:* You can prove the statement when $n = 1$ first. The proof for general values of n is similar.)

33.3-2
Prove Lemma 33.7.

33.3-3
Prove equation (33.29). (*Hint:* The proof for $n = 1$ dimension is straightforward. The proof for general values of n dimensions follows along similar lines.)

33.3-4
Show that the function f in equation (33.32) is a convex function of the variables $w_0, w_1, \ldots, w_n$.

33.3-5
Compute the gradient of expression (33.33) and explain how to evaluate the gradient in $O(nm)$ time.

33.3-6
Consider the function f defined in equation (33.32), and suppose that you have a bound $\|\mathbf{w}\| \leq B$, as is considered in the discussion on regularization. Show that $L = O(B)$ in this case.

33.3-7
Equation (33.2) on page 1009 gives a function that, when minimized, gives an optimal solution to the k-means problem. Explain how to use gradient descent to solve the k-means problem.

Problems